CHAPTER-1

INTRODUCTION

Blockchain technology has emerged as a very transformative solution by introducing decentralized, transparent and secure digital transactions. While popular blockchain networks like Bitcoin and Ethereum work well without causing any issues in basic ledger functionalities, they remain inefficient in their ability to provide use case specific transaction requirements.
Existing conventional networks treat all transactions equally or rely highly on gas-fee-based prioritization mechanisms. Either of the models create opportunities for unfair transaction prioritization and also allows a few users to manipulate the transaction execution order by offering higher fees disregarding the actual importance of an individual transaction.

This project, titled “Optimised Blockchain Network for Use-Case Specific Transaction Prioritization”, proposes a model that is designed to solve the existing limitations in the blockchain systems. The primary objective is to introduce an optimised prioritisation mechanism based on the individual function of the transaction while performing all the basic functionalities of a conventional blockchain network. By sorting each transaction type such as swapping, borrowing, lending and staking, to a specific predefined priority level, which allows the network to interpret the priority of the transactions which is absent in the traditional blockchain networks.

The project is implemented using the Hyperledger Fabric Framework, improved with a custom transaction schema and a priority based ordering service. The custom schema provides a structured transaction classification. To prevent extensive delays or complete starvation of lower priority transactions, reserved block slots are integrated. The system uses a single channel network architecture to achieve visibility and reduce complexity.

Integrated dApps initiate the transaction with a type, automatically setting the priority which is read by our custom ordering service to prioritize the transactions that are meant to be prioritized. By eliminating the reliability on gas-fee for transaction prioritization and replacing it with a more logic driven solution this project establishes a more reliable and fair blockchain network.

CHAPTER 2

PROBLEM DEFINITION

Conventional blockchain networks are designed for general-purpose use, which means it will end up leading to inefficiencies while working on specific requirements such as prioritizing certain types of transactions. These inefficiencies lead to higher transaction costs, longer transaction processing time and reduced throughput for critical operations.

This project eliminates these limitations by developing a blockchain architecture that ensures all the standard and conventional ledger functionalities like transaction verification and recording but also will use customized rules to prioritise specific transactions. This approach will help us eliminate the limitations and unwanted resources being used.

Proposed Solution :

1) A custom transaction schema which is capable of categorising transactions based on their type (staking, lending, borrowing, swapping, etc.).
2) A custom ordering service that prioritizes transactions based on the type specific priorities pre-defined, before passing them on to the actual consensus mechanism
3) Use of RAFT as the main consensus protocol within hyperledger fabric for efficient fault tolerance and validation of the blocks.
4) Fair inclusion of all transactions, eliminating frontrunning and unnecessary congestion throughout the process and smart block formation.

CHAPTER 3

LITERATURE REVIEW

3.1 An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends

3.1.1 Introduction

This paper discusses all the basics of Blockchain Technology, it initially explains all the basics of blockchain technologies and all the key characteristics like decentralization, persistence, anonymity and auditability. It also presents the different types of consensus mechanisms like PoS, PoW, PBFT and more; each in depth talking about each of their pros and cons and the paper ends with a brief discussion about challenges and problems faced like scalability storage optimisation, privacy leakage and selfish mining.

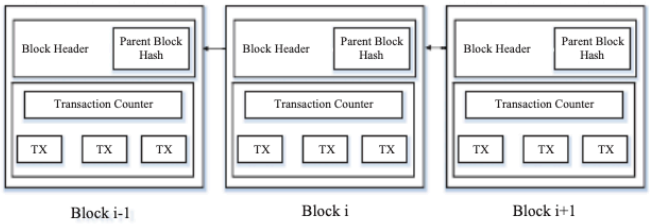

Fig. 1: An example of blockchain which consists of a continuous sequence of blocks.

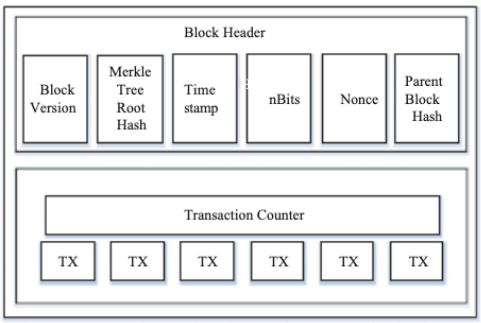

Fig. 2: Block structure

3.1.2 Methodology

- This research paper implements methods emphasising on blockchain consensus algorithms like Proof of Work (PoW), Proof of Stake (PoS), Practical Byzantine Fault Tolerance (PBFT).
- Each of the methods have been discussed individually exploring each of their advantages and disadvantages.

3.1.3 Relevance to our project

This paper explains how Blockchain Architecture works, the key characteristics of a blockchain and also the problems faced in the current blockchain industry. The paper initially helped ideate the basic structure for this project. The paper then tells about the different types of consensus mechanisms in depth discussing the pros and cons of each one of them which helped decide the type of consensus mechanism to be used in this project.

3.2 Latency Analysis for Raft Consensus on Hyperledger Fabric

3.2.1 Introduction

This paper provides a performance characterization of Hyperledger Fabric (1.0) by conducting various experiments to evaluate its throughput, latency, and scalability, by introducing various custom benchmarks to test the fabric. This paper explores the effects of transaction workloads changes and different chaincode parameters on the performance of the hyperledger fabric.In conclusion, it is seen that the hyperledger fabric performs well, with low transaction latency, but this performance slowly decreases as the load increases due to scaling as shown in the experiments.

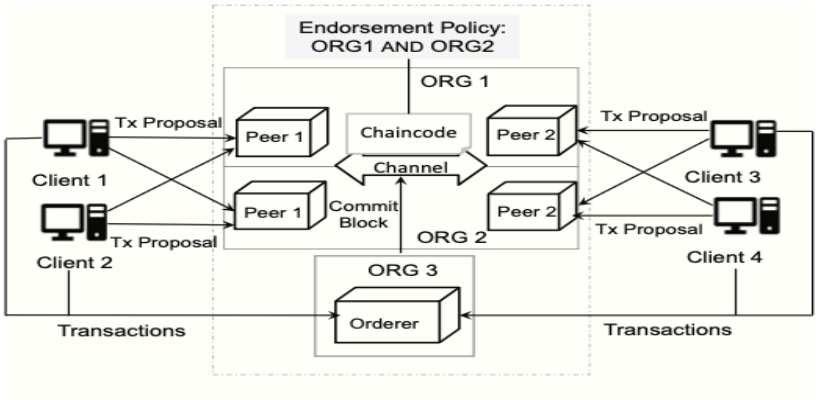

Fig Experimental Setup

3.2.2 Methodology

In this paper, the performance of the hyperledger fabric (1.0) is spread out across four different methods.Initially the paper characterizes the throughput and latency of the hyperledger fabric, by an extensive experimental setup.Then it defines a set of micro-benchmarks that are custom built for the fabric, which are used to study the fabric's performance characteristics. This section describes different types of the transaction such as Read, Write, Mixed, Null and smart contract(also called chaincode in hyperledger fabric), which affect the transaction latency.

Further there is an experiment to test the scalability, which shows how the fabric performs when the chaincode is scaled, by increasing the number of channels or by increasing the number of peers within a particular channel.

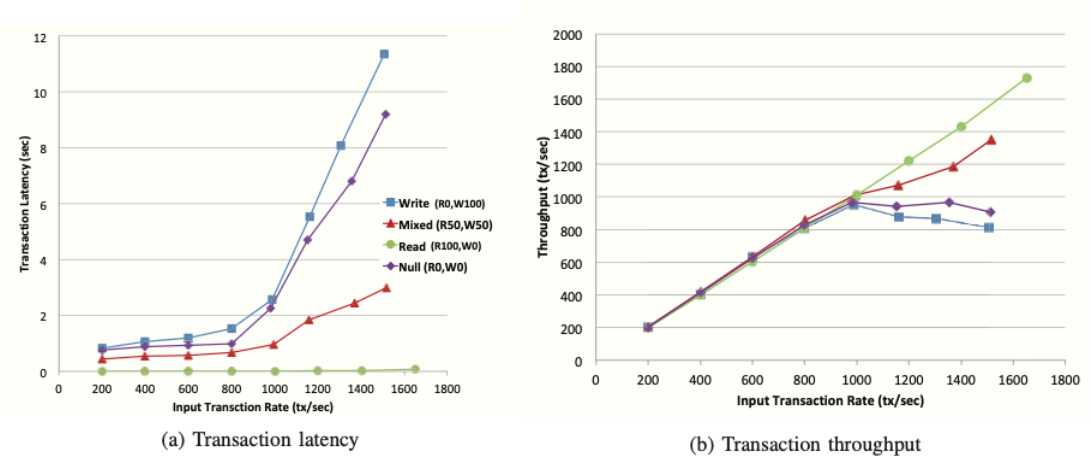

Fig Latency and throughput measurements for varying input load

3.2.3 Relevance to our Project

The paper presents a structured experiential approach to characterize the performance and scalability of the Hyperledger Fabric 1.0 blockchain platform. It showcases how throughput and latency are affected by different configuration changes.

The version of hyperledger fabric tested in this paper(1.0) does not support parallel transaction processing, which limits the performance of the ledger during higher workloads or in case of scaling. The presence of endorsements in this proposed methodology ensures safety, but the time taken to process each transaction(latency) increases because of this.
Having fixed batch sizes affects the performance and hence it is suggested that the batch size and load should be dynamically selected.

We use these insights from the findings and optimize block formation speed and ordering configuration to fine-tune our priority-based block creation in order to achieve optimized transaction sequencing.

3.3 Latency Analysis for Raft Consensus on Hyperledger Fabric

3.3.1 Introduction

This paper investigates the performance of the RAFT consensus mechanism specifically within the Hyperledger Fabric ecosystem. This paper focuses on the Raft consensus latency that is time consumption of consensus, it provides a theoretical model of the Raft consensus latency. Subsequently, the authors analyze the effect of consensus latency on the transaction latency and the effect of a block size on consensus latency by extensive experiments.The experimental result shows that the proportion of consensus latency in the entire transaction latency is relatively small, while the block size has more greater effect on the consensus latency

3.3.2 Methodology

The authors measured the end-to-end latency using benchmarking tools like Hyperledger Caliper. The testing environment included a few peers, endorsers, and an ordering service configured in a Raft cluster. Transactions were sent at different rates to measure the latency and to record various factors such as Proposal phase delay, Ordering delay and Commit delay. They tested different configurations by altering BatchSize, BatchTimeout, and number of peers to measure the performance changes.

3.3.4 Relevance to Our Project

The paper provides key information into how Raft consensus latency process affects the transaction latency and the effect of a block size on consensus latency contributing to overall transaction latency. The experimental result shows that the proportion of consensus latency in the entire transaction latency is relatively small, while the block size and batching configurations have a greater effect on the consensus latency. The study also identifies some shortcomings such as High latency under load,SPFO,lack of transaction prioritization, and performance degradation under scale which are critical challenges. Our project look to use these insights and address these limitations by exploring ideas such as priority-based transaction ordering layer to ensure faster execution for critical transaction like staking, lending and borrowing

3.4 Raft Consensus Mechanism and its Applications

3.4.1 Introduction

The paper explains the working of Raft consensus algorithm which is a leader based mechanism used to achieve distributed consensus. It serves as a substitute for Paxos designed to be easier to understand and implement while achieving the same level of reliability. Raft maintains data consistency throughout the network by managing replicated logs among dispersed nodes.The authors explain how Raft achieves log replication and consistency across the nodes.

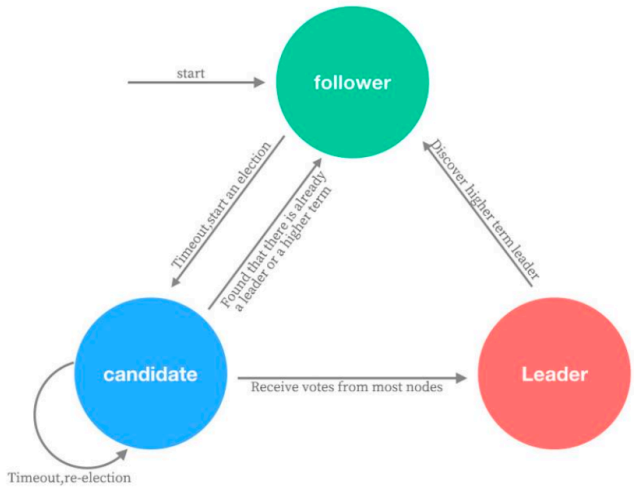

Fig Transition Relation between three states

3.4.2 Methodology

There are three key states of RAFT - Follower, Candidate, and Leader - mentioned in this paper and explains how the nodes transition between these states. Some of the topics mentioned are the leader election process, log replication, and the use of heartbeats for synchronization. The authors use a step-by-step approach to explain how Raft ensures consistency, even in cases of network partitioningRaft maintains log consistency by checking the index and term values of the appended entries. In the event that mismatches arise, it truncates the follower's log and overwrites it with the leader's copy to ensure a uniform state among all nodes.

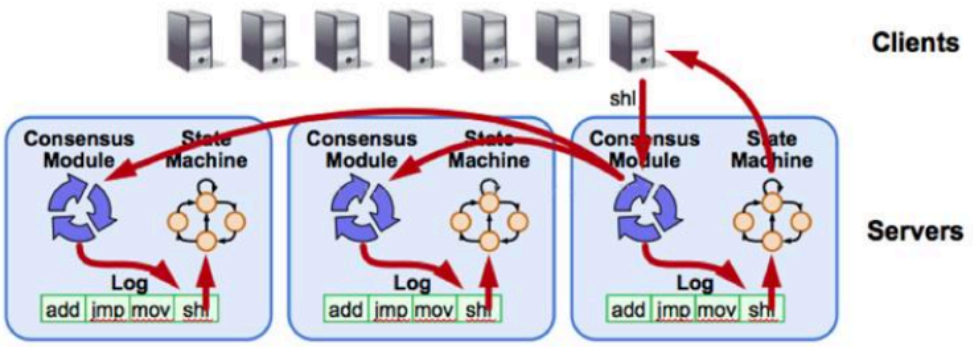

Fig Consensus Process

3.4.3 Principal Characteristics and Benefits

Simplicity: simpler than Paxos in terms of implementation and reasoning.
High Fault Tolerance: able to withstand node failures up to ⌊(n-1)/2⌋.
Election of Decisive Leaders: helps decide one single leader.
Scalability: lets you compact logs and replicate them in modules.

3.4.4 Application of Use

Raft is used by permissioned Blockchain like Hyperledger Fabric and Consul. These systems must come to a decision quickly and must be available all the time. Raft is used as the foundation for our project's ordering system. The system sorts and prioritises transactions, and it keeps working even when the network goes down.

3.4.5 Relevance To Our Project

Raft is essential in our permissioned blockchains in achieving ordering transactions based on priority. Although Raft's functionality is thoroughly explained in the study, possible scalability problems when applied to large-scale networks are not examined.The study does not give empirical data or benchmarks comparing the RAFT consensus with any other consensus mechanisms in actual distributed systems; instead, it concentrates on the theoretical underpinnings of the Raft algorithm without carrying out comprehensive experimental performance evaluations.. We look to build upon this by deploying Raft on a Hyperledger Fabric test network and assess it using actual workloads. We will perform empirical benchmarking and measure latency, block time, and transaction success rate under different workloads

3.5 A Case Study On Solana

3.5.1 Introduction

Solana is a very high performance Layer-1 Blockchain which is known for its fast transaction processing capability and scalability. It is multiple times faster than its competitors due to its unique approach of using a Proof of history (POH) mechanism, which is just a Proof of stake consensus mechanism coupled with timestamps associated with each block being sent in by the nodes. It was designed to face the issues related to throughput and latency limitations of the existing traditional blockchains such as Ethereum.

3.5.2 Methodology

- Tower BFT is Solana's consensus mechanism built on top of POH, which enables validators to vote on the state of a ledger at a given point of time, which then helps reduce the network overhead.
- Usage of Turbine and Gulf Stream, where turbine splits the data for a quicker propagation of blocks, while the Gulf Stream allows Solana to forward transactions to validators before the next block is even produced.
- Sealevel is Solana's way of implementing 'parallel smart contracts', which helps improve the efficiency of executions.

3.5.3 Relevance to our project

- Seeing Solana's Tower BFT and gulf stream methodology which removes traditional mempool delay helped us decide to create a custom ordering service with external validation plugin to reorder the transactions more intelligently in Hyperledger Fabric.
- Solana is very highly used in Defi (swaps, lending, staking). Even this project heavily emphasis on Defi centric transactions such as swap, borrow, lend, transfer. This study on Solana helped us to understand that why DeFi transaction types require faster ordering, the expected load patterns in the financial blockchains and the importance of fairness to avoid front running.

3.5.4 Gaps covered

- Solana emphasis on speed because of which low fee transactions may get delayed or even dropped. But we ensure reserved block slots for low-priority transactions which helps maintain fairness.
- Even though Solana is highly optimizable it does not provide configurable or use case specific transaction prioritization like this project.

3.6 A Case Study On Polkadot

3.6.1 Introduction

Polkadot is a Proof of Stake(PoS) Layer 0 blockchain protocol designed to enable cross chain communication between multiple blockchains.It uses a hub and spoke architecture to connect multiple blockchains with each other. The hub is what is called the relay chain and the other blockchains are called parachains. The main token DOT operates on the relay chain where the consensus is being achieved. dApps function individually on parachains but are finally received by the relay chain to sync up. Polkadot allows the developers to create blockchains using their own logic and rules, which makes it suitable for multi-use-case blockchain ecosystems.

3.6.2 Methodology

- Polkadot's core chain is called Relay Chain which is responsible for network security, consensus and also the cross-chain communication. Multiple parachains then connect to the Relay Chain which executes computations in parallel which significantly improves scalability compared to the single chain networks.
- Use of NPos as its consensus mechanism, where the nominators select the validators they trust with their stake, which improves decentralization while also ensuring efficient block validation with security.
- Substrate framework allows the developers to build custom blockchains. It is a modular framework which already includes components like consensus, mempool logic, smart contract modules and governance. This reduces the development time while still providing domain specific customisation.

3.6.3 Relevance To Our Project

- Polkadot's modular substrate approach inspired the custom logic modules like external validation plugin and the priority-based ordering layer being used in this project. This ensures the network is highly adaptable and scalable in future extensions.
- Polkadot enables the customizable chain logic, deploying an entirely new parachain which makes it more complex. While this project introduces a more efficient, lightweight prioritization mechanism within an existing blockchain network, which is more practical and efficient.

3.6.4 Gaps Covered

- In Polkadot parachains focus on scalability which may cause the nodes to prioritize high value and complex transactions leading to delay or even dropping of the smaller transactions. But by implementing a reserved block slot mechanism in this project it allows even the lowest priority transactions to get included within time.
- Polkadot does not define how a mempool ordering should be handled. Without prioritization urgent financial transactions might get delayed due to congestion. By redesigning the mempool to sort, batch and order transactions based on priority this project ensures faster processing of critical transactions.

CHAPTER 4

PROJECT REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

ID	Requirement Name	Description & Priority
FR1	Transaction Submission	Handles transaction input, validation, and rejection of invalid entries. (High Priority)
FR2	Priority Assignment	Automatically assigns fixed priority levels based on transaction type. (High Priority)
FR3	Mempool Management	Maintains a priority-ordered transaction pool with size limits and status API. (High Priority)
FR4	Batch Processing	Groups transactions by time or count triggers for blockchain submission. (Medium Priority)
FR5	Dynamic Gas Fee Calculation	Calculates variable gas fees using priority, congestion, and base rate factors. (Medium Priority)
FR6	Wallet Management	Manages wallet creation, balances, transaction history, and token storage. (High Priority)
FR7	Transaction Processing	Validates balances and executes type-specific logic for all transactions. (High Priority)
FR8	Transaction Status Tracking	Tracks transaction lifecycle and emits events for state changes. (Medium Priority)
FR9	REST API	Provides standardized API endpoints for client interaction and data access. (High Priority)
FR10	Monitoring & Metrics	Tracks performance metrics and logs events for system observability. (Medium Priority)

4.2 Assumptions and Dependencies

Assumptions :
The system assumes that the network environment will be stable and reliable with less than 1% packet loss, ensuring smooth communication between the orderer and the peer. All the nodes are synchronised through the NTP, maintaining consistent timings for transaction ordering as well as validation. The types of transactions(swap, borrow, lend, transfer) and their priorities are well defined and properly established across the network. Token economics including token valuation as well as inflation control are managed externally to the system. It is also fairly assumed that the users are aware of the transaction priorities as well as the gas fee implications before they submit their transactions.

Dependencies :
The entire implementation relies on the availability of hyperledger fabric's binaries and docker images that handle the network setup and containerization. The certificate Authority is required to be functioning properly and is essential for managing the identities while enabling secure and authenticated communication. The deployment of the entire network depends on Docker and Kubernetes container platforms for scalability as well as lifecycle management. It is also necessary to integrate with the cloud providers APIs for hosting and managing the production environments. Additionally, fabric SDK and GO standard libraries are required by the system to support the core functionalities of blockchain and its interactions.

CHAPTER 5

SYSTEM DESIGN

5.1 Environment and Network Setup

The hyperledger fabric framework was installed and deployed on a local environment to establish a controlled blockchain network. The setup process involved obtaining the required docker images and binaries, configuring hyperledger fabric's CLI, and creating the necessary cryptographic material using fabric's certificate authority. Each peer, orderer, and channels for these members was configured on the system to emulate a permissioned blockchain environment.

Setting up hyperledger fabric on a local system involved resolving compatibility issues between docker containers and the host networking, the chaincode life dependencies, and configuring the TLS certificates of each of the peers and orderers to ensure secure communication. A conscientious amount of configuration of the environment variables concerning the peers regarding their addresses and the orderer's endpoints was essential to ensure that these peers could communicate over secure channels and that smart contracts could be instantiated correctly on the ledger itself.

A channel was created and joined by all the relevant peers, once the network components were configured. These channels act as a shared communication layer wherein the transactions are endorsed, ordered, and committed. All of this came together only when the certificate authority, orderer, and peer containers in a predefined sequence started, as the deployment of the test network, which ensured that the network deterministically bootsrapped and could support subsequent chaincode operations.

5.2 Smart Contract design and execution framework

The smart contract layer is one that lies at the core of this framework, developed using Go and Fabric Contract API. This component governs asset creation, state transitions, and the priority based transaction execution. The contract further introduces an extensible structure for custom wallet management and transaction handling, which helps provide atomicity and integrity guarantees, through fabrics own endorsement policies and validation mechanisms, using RAFT.

A wallet in the network is represented as an object containing its balance, transaction history, and associated tokens. Global uniqueness and Tamper resistance of a new wallet created is ensured by using a unique cryptographic address, generated using a combination UUID and SHA-256 hashing. There is also a functionality to verify the existence of wallets, serialize and deserialize wallet data, and perform balance checks with the help of high precision arithmetic using floating point operators.
The smart Contract supports four fundamental transaction types : swap, borrow, lend, and transfer, each of which is assigned a priority level that decides the order in which the transactions are processed. 0 is assigned to swap as the highest priority, whereas 3 is assigned to transfer as a lowest priority. These operations cover various financial actions from, direct transfer of funds from one account to another, to inter-wallet lending, each of which maintain a consistent transaction structure which include identifiers, timestamps, and contextual data. All of the transaction records are logged into the wallet's history, which helps establish immutability in the transactions records, that helps maintain transparency and verification.

Consistency in the data is maintained as the contract enforces atomic state updates for both the sender and receiver wallets, which are modified within the same transaction scope, ensuring that failure during the execution of any transaction would result in a complete rollback. This ensures ledger integrity across all the peers.

5.3 Channel Architecture and Design

The proposed system design and implementation adopts a single channel architecture rather than a multi-channel one. Conventional Hyperledger fabric deployments utilize multi channels to isolate the data among organizations. However, a unified single channel was implemented to maintain global visibility, and facilitate cross organizational transaction prioritization.

A single channel ensures that all transactions, regardless of its origin or participating peers, will enter a common ordering pipeline. This uniformity in the structure is crucial to this priority based transaction model, as it allows the ordering service to evaluate and reorder all the transactions across the network, based on the predetermined priorities, rather than isolating these transactions within separate channels or domains. This reduces the interchannel synchronization complexity and ensures fairness in transaction sequencing throughout the entire network.

5.4 Transaction prioritization and mempool Formation

A transaction submitted by a client into the network is first collected into a mempool, which is a transient holding area that buffers unconfirmed transactions which are to be placed into blocks. Each incoming transaction includes its origin, destination, transaction type, alongside other metadata like timestamps and the computed priorities.

The mempool dynamically maintains the statistics such as total transaction count, pending transactions distribution across priorities, and the maximum mempool capacity. This allows the network to regulate inflow and monitor the backlog of the transactions in real time. Each of the incoming transactions is assigned a priority, which is predetermined from its type : swap being the highest in priority (0), followed by borrow, lend, and transfer being the lowest priority (3). This categorization helps in the prioritization of the transactions by the ordering service before block formation.

During the block assembly, a batcher module periodically selects transactions from the mempool. A block can accommodate a predefined number of transactions (5 per block), which are based on the FIFO(first in first out) policy, combined with the prioritized sorting of the transactions. Transactions that arrive earlier are considered first, but within each of the epochs, higher priority transactions are advanced to the front of the selection queue.

To address the potential issue of starvation, wherein the lower priority transactions might indefinitely wait due to a continuous influx of higher priority transactions, a quota or reservation system is used. This ensures that every block reserves a specific number of slots for lower priority transactions, which helps in preserving fairness and preventing the persistent backlog accumulation. This balance between the throughput and fairness allows the network to simulate real world resource allocation mechanisms.

5.5 Transaction lifecycle and state transition

Once transactions are selected from the mempool, they undergo endorsement by the designated peers and are then submitted to the ordering service. The ordering service then arranges the transactions according to the predetermined priority logic before these transactions are grouped and sent to form a new block. The block is then subsequently distributed to all the peers on the channel, where the validation and commitment occurs.

Each of the transaction records transitions through various states which include, pending, processing, completed, or failed, which reflects the transactions progression in the system. These states are managed by the internal transaction structures, ensuring clarity in tracking execution status. Additionally transaction failures are logged for diagnostic purposes, and may be retired or analysed to improve batching or resource allocation.

5.6 Congestion Based Gas Fee Allocation

The gas fees on the network is influenced by two main factors which is, network load - the number of active transactions that are queued on the mempool and the type of transaction that is being sent by the user. The amount of tokens on the transactions plays no role in determining the gas fees hence resulting in a very efficient system that does not incentivise users with higher or lower token amounts. The gas fees are dependent on the congestion factor which is derived from a sigmoid function which helps in attaining a very smooth gradient increase in gas fees as the congestion on the network increases. The transaction type also provides a multiplier to the already existing base fee depending on the priority of the transaction type which directly plays an influence on where the transaction gets ordered in a block hence providing privilege to high priority transactions. The base fee is multiplied with the congestion factor obtained from the sigmoid function which is then multiplied with the transaction type multiplier to attain the gas fee for the transaction. This system provides an efficient economic balance of tokens while also maintaining good profitability to the network.

CHAPTER 6
PROPOSED METHODOLOGY

6.1 System Architecture Overview

The proposed system is a layered architecture system which follows an approach where the client submits transaction requests to the priority gateway where it undergoes processing, it is then submitted to the blockchain layer which is Hyperledger Fabric helping to run the transactions through consensus for validation from peers. The transaction is then registered onto the ledger to store the state permanently.
This approach consists of three main layers:

The Client Layer: Applications on the internet submit transactions from users with different transaction types which have different priorities on the proposed network. Applications could submit one of four transaction types - swap, borrow, lend and transfer.

The below table shows the priority number for each transaction type:

TrxType	Priority Number
Swap	0
Borrow	1
Lend	2
Transfer	3

Each transaction type is linked to a priority number which is later used in the next layer to determine the priority that the transaction holds during batching on the network.

Priority Gateway Layer: The priority gateway layer is a custom middleware service that helps to intercept transactions, order transactions based on their priority type and also helps in managing a priority-based mempool. It is also the layer responsible for performing batch processing before submitting transactions to the blockchain.

<u>The Priority Gateway Layer consists of mainly two logic layers:</u>
<u>Mempool</u>: The mempool is an array which could be called a queue in simple terms which holds the transactions in a queue before they get processed to get registered on to the ledger. The mempool has a window size which it considers to perform reordering of transactions to package them into blocks. The window size influences the

TPS(transactions per second) or throughput of the network as this determines how many transactions are sent per second for validation. The window size is not limited to the number of transactions but is also influenced by a time factor where a block gets formed either with respect to the max number of transactions allowed per block or through elapsing of a certain time period. The mempool works based on the approach of whichever happens first, either the time period elapsing or the maximum number of transactions per block getting filled.
Batcher: The batcher works hand-in-hand with the mempool to see when a window gets completed in order to take the transactions out of the mempool to create blocks out of them using a quota based fairness strategy which prevents starvation of any particular type of transaction. The batcher also alternates between using this quota based approach and FIFO approach for taking in transactions inside of a mempool window.

Blockchain Layer: The blockchain layer consists of Hyperledger Fabric's test-sample network which delivers a template of a RAFT based consensus blockchain network where the batcher feeds blocks that have been prioritised to be able attain consensus in order for the blocks to register on to the ledger and become a permanent state. The Blockchain Layer is accessed by the Batcher through the fabric client sdk which lets the Batcher invoke calls to Fabric's internal logic run functions like adding blocks to consensus, creating wallets, minting tokens etc. The blockchain layer also executes smart contracts for wallet operations and helps in maintaining the distributed ledger.

This system of three layers operates on a hybrid architecture where transaction creation and ordering occurs off-chain while consensus and ledger management is handled by Hyperledger Fabric's native RAFT consensus.

6.2 Priority Assignment Methodology

Priority assignment follows a deterministic mapping approach based on the type of transaction.

Priority - 0 (Highest) - Swap Transaction: A swap transaction is a transaction which basically indicates an exchange of tokens in a blockchain network. This would have the highest priority in order to attain fast order routing and fulfillment. Small time gaps due to transaction processing could result in MEV attacks and front running.

Priority - 1 - Borrow Transaction: These transactions let users borrow different tokens while keeping their current token holdings as collateral. Borrowing could be time sensitive for accessing liquidity but is less critical compared to swaps.

Priority - 2 - Lend Transaction: This is the other side of a borrow transaction where users get to lend their tokens to earn interest from the borrowers. A high priority isn't necessary in this case as these transactions can tolerate slight delays. No real loss is being incurred due to delays in these transactions.

Priority - 3 (Lowest) - Transfer Transaction: These are simple transactions executed on a peer-to-peer basis where there is no time sensitivity. Transfers can take their time in processing as users never incur a loss in any case while considering a normal transaction.

The priority gateway automatically extracts the priority out of the transaction type, ensuring smooth and tamper-proof priority assignment and ordering of transactions. This prevents the users from arbitrarily inflating priority levels while maintaining fairness through algorithmic determination.

6.3 Mempool Management Algorithm

The mempool is integrated as a min-heap priority queue where transactions are ordered considering two main things which is the priority (ascending 0 - 3) and FIFO(First In First Out). The mempool holds a window size which determines the number of transactions that are considered for block creation. The window size influences the throughput of the network. New transactions get added to the mempool on FIFO basis and then get selected into a window which might consist of a set number of transactions with any combination of priorities. The mempool then uses an alternating quota based approach to order the transactions inside of a block to create priority based on the type of transactions.

The alternating quota based approach stacks transactions into a block in two different ways as the name says because it is alternating. The transactions get placed in ascending order of priority in a FIFO approach within the window size on one block whereas the next block will arrange the transactions in a quota based manner. The quota based approach is an anti starvation strategy in order to maintain that no one priority number of transactions gets filled in a single block creating a starvation of lower priority transactions.

Quota-Based Anti-Starvation

Mempool Window Size: 5

Quota per priority: A block must contain at least one transaction of a priority level if available.

Example:
Mempool: [0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 3]
Batch 1: [0, 0, 0, 1, 2] - Ensures mix of priorities
Batch 2: [0, 0, 0, 0, 1]
Batch 3: [0, 0, 3]

From the above example where priority 0 transactions are dominant in the initial part of the mempool there is a possibility that the transactions with priority of 1, 2 and 3 get starved until the window comes to them after the transactions with zeros are processed. To prevent this, the anti-starvation approach uses the strategy to contain at least one transaction of a priority level if available so that higher priorities do not get starved until lower ones get processed for a very long time on big window sizes and mempools.

The mempool management algorithm consists of functions like Insert which helps to insert transactions as they are created by users into the mempool. The mempool also has functions that help it to coordinate with the batcher and send messages to the batcher to intimate it about the type of transactions and blocks it is receiving, which may be a pure priority based or quota based anti starvation block because the system alternates between these two types of blocks. It contains a delete function which is used to delete transactions from a block if they fail.

6.4 Batching Strategy

The transaction batching strategy implements a dual trigger mechanism with two independent conditions as mentioned in the above mempool management algorithm where batch creation depends if either of two conditions listed below are satisfied:

Trigger 1: Count Based - A count based trigger is a flat integer that the batcher considers to create a block. It can be defined as the maximum number of transactions that the batcher is ready to accept inside of a block. A block is created by the batcher if the condition of a said number of transactions has filled a block.

Trigger 2: Time Based - A said amount of time elapsing, also called an epoch plays an influence in the creation of blocks. The number of transactions that have entered a mempool in a defined amount of time are considered for batching and creation of blocks.

The above triggers work together in a way that whichever happens first is taken into effect. With an example of 200 transactions on the count based trigger and 3 seconds on a time based trigger, a block gets created by the batcher when one of these conditions is satisfied. In cases where the epoch(3 seconds) has elapsed, the block creation happens regardless of the number of transactions that have accumulated which may be < 200 transactions. In cases where the count of 200 transactions have been achieved before an epoch elapses, a block creation still happens without taking the amount of time that has elapsed which may be < 3 seconds.

The batcher holds code to interact with Hyperledger Fabric via the Gateway API which uses the Fabric Client SDK in order to invoke calls on Hyperledger Fabric's internal logic.
The batcher receives ordered transactions from the mempool in order to combine them to satisfy one of the above mentioned trigger mechanisms.

The batcher also while delivering the above functions helps in monitoring and statistics collection through functions like batcher stats and detailed stats that return basic and extended runtime info which could be further used in data dashboard building which would be essential in knowing key statistics of the functioning of the whole off-chain ordering service.

6.5 Dynamic Gas Fee Calculation

The gas fee calculation in the network purely depends on the load on the network and the type of transaction that the user creates. The gas fees increase as the priority of the transaction increases (0 - high, 3 - low). Lower gas fees are used by transaction types like transfer and lend which do not have time sensitivity and hence do not have the constraint to be executed quickly. A lower gas fee is charged for transactions like these as it is not really a prioritised transaction which is getting privileges over other types of transactions. Higher gas fees are used by high priority transactions like swap and borrow which are very time sensitive as swap may be liable to front-running without high speed of transaction execution and borrowing may be affected with quick access to liquidity when it is available.

To efficiently deliver the above gas fee model which is influenced by both network load and transaction type, the use of a sigmoid function gives a smooth gradient of gas fee for each type of transaction and the amount of network congestion that exists during the execution of that particular transaction type. The gas fee is calculated with a base fee

plus additional fees that are influenced by the above factors making use of the Sigmoid function.

Sigmoid Function Formula:

Gas Fee = Base Fee x Priority Multiplier x Congestion Factor(sigmoid function)

Priority Multiplier:

Transaction Type	Priority	Multiplier
swap	0	4x
borrow	1	3x
lend	2	2x
transfer	3	1x

Where,
Base Fee = 0.001
Priority Multiplier = {4.0, 3.0, 2.0, 1.0} for priorities {0, 1, 2, 3}
Congestion Factor = 1 + (MaxSurge / (1 + e^(-k(u - 0.5))))

- u = utilization = current_size / max_size
- MaxSurge = 10.0 (maximum additional multiplier)
- k = 8.0 (steepness parameter)

The sigmoid function provides a very gentle fee increase during low congestion and strong economic signals with a good steep increase during medium congestion on the network. When the network is at 75-100% congestion it has a natural fee ceiling which prevents excessive costs.

The above fee structure creates economic incentives for submission of low priority transactions during off-peak times and self regulates network load through price discovery.

Utilization	Congestion	Swap	Borrow	Lend	Transfer
0%	1.18x	0.00472	0.00354	0.00236	0.00118

10%	1.39x	0.00557	0.00418	0.00278	0.00139
25%	2.19x	0.00877	0.00658	0.00438	0.00219
40%	4.10x	0.0164	0.0123	0.0082	0.0041
50%	6.00x	0.024	0.018	0.012	0.006
60%	7.90x	0.0316	0.0237	0.0158	0.0079
75%	9.81x	0.03923	0.02942	0.01962	0.00981
90%	10.61x	0.04243	0.03182	0.02122	0.01061
100%	10.82x	0.04328	0.03246	0.02164	0.01082

The above table shows the increase in gas fees as the congestion and priority increase.

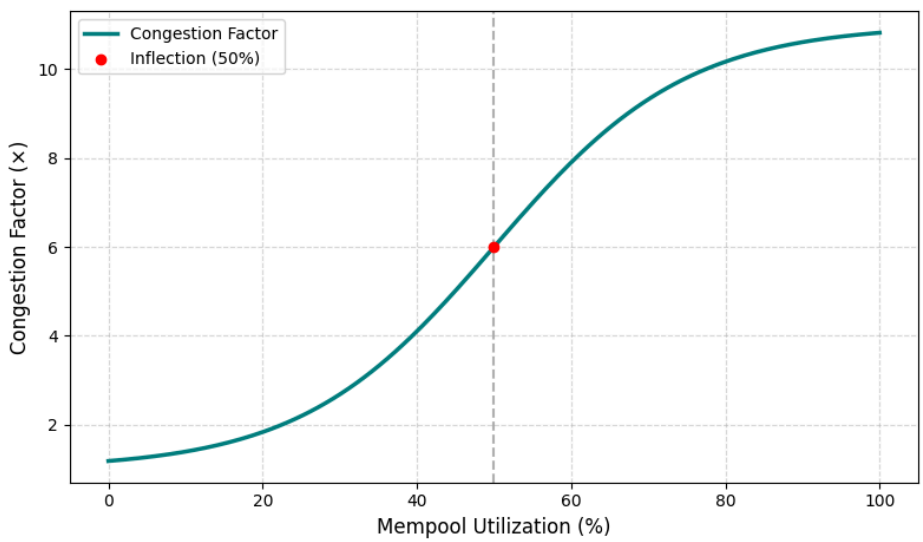

Fig Congestion Factor vs. Mempool Utilization

6.6 Smart Contract Architecture

The smart contract, or chaincode as in Hyperledger fabric is designed to manage the transaction processing between wallets, ensuring secure, transparent, single channel, and efficiently ordered transactions based on the predefined priorities. Each wallet in this case is represented as an object or a structured data model which has a unique address, a base token balance that is required as per the gas fees, transaction history, cumulative values of the total amounts lent or borrowed, and a count of the additional tokens that the user holds.

Each transaction is recorded with the required metadata, which includes the unique transaction ID(TxID) for each transaction, the source and destination wallet addresses, transaction amounts, the type of transaction (which could include swap(highest priority {0}, borrow{1}, lend{2}, or transfer(lowest priority {3}), assigned priority levels that depend on the type of the transaction, and the precise timestamps.

The core functionality of the system is implemented through the set of smart contract functions like CreateWallet() for generating new wallets, ReadWallets(address) for the retrieval of states, Transact(from, to, amount, type) for the execution of transactions after applying the priority based ordering logic, PutWallet(wallet) for the updates to the ledger, and GetPriorityByType(type) to determine the priority that the transactions have to be processed in.

During the execution of the transaction, the contract is used to dynamically calculate the gas fees based on the current mempool size and a multiplier based on priority, verifying sufficient balances, and applying specific operations based on type. It then updates both the sender and the receiver balances, records the transaction in the history of their wallets, and commits all the changes to the ledger. The database required to store all of this information will include Hyperledger Fabric's CouchDB state database, which will allow for efficient querying, storage, and retrieval of wallet data as JSON documents ensuring reliability in the decentralized financial processing.

6.7 Integration with Hyperledger Fabric

The System design integrates with blockchain networks using the Hyperledger Fabric Gateway API(v2.5 {2.x}) to facilitate the streamlining of client interactions and transaction management. In its current test configuration, the network consists of two organizations, each consisting of two peers, primarily set up in order to test the functionality and performance of the Raft based ordering service. However, the architecture is not limited to just two organizations or two peers per organization and can be scaled up, in fact provides better and more reliable decentralization only when scaled up to multiple organizations and numerous users within each organization.

This network operates on a single channel called mychannel and uses the started CouchDB state database provided by hyperledger fabric for the queries regarding state. The integration process begins with the establishment of a connection, wherein the client provides their connection profiles including identity credentials (private keys and certificates) to form a secure gRPC connection to the fabric Gateway.

During transaction submission, the transactions that enter the mempool are initially sorted according to priority and the time at which they arrive, in each batch. These ordered transaction batches are initially serialized, then endorsed through the gateway API, and finally submitted to the orderer who is responsible for the consensus, before awaiting the confirmation of the committing of the block through consensus from the peers.
The event listening mechanism constantly waits for events occurring through the chaincode, monitors the block commits, then updates the status of the transactions from pending to completed, and finally notifies the client up completion of the processing. This flow in the consensus ensures reliability and determinism. The priority gateway is used to submit transactions based on the predetermined order of priority, these are then endorsed by the peers, sequenced into blocks by the orderer, and finally committed to the ledger after validation by the peers.
Hyperledger Fabric helps in ensuring scalability, fault tolerance, and deterministic state across the distributed ledger by leveraging RAFT consensus mechanism, while implementing the novel design involving prioritized transaction handling.

6.8 Implementation Technologies

Category	Components / Tools
Programming Languages	Go (1.21+): Priority Gateway implementation; Go: Smart contract (chaincode) development
Frameworks & Libraries	Hyperledger Fabric 2.5.x; Fabric Gateway SDK (Go); Fabric Contract API; Gin/Echo (REST API); container/heap (priority queue)
Infrastructure	Docker & Docker Compose; CouchDB; React Js
Development Tools	fabric-samples/test-network; peer CLI; Go modules; Git

The network system makes use of the above technology stack which helps in ensuring efficient priority based transaction processing. Golang is used as the primary language for both the gateway as well as the development of the smart contract due to its established efficiency and concurrency support in other use cases. The underlying blockchain framework is provided by Hyperledger fabric, while docker and CouchDB are used to manage the deployment as well as maintain the deterministic state of the ledger. The visualization of the metrics and processing of the transactions will be seen on a react app and frontend that can be used to verify the ordering and the processing of the transactions.

6.9 Testing Methodology

For this particular system, the testing framework was designed to ensure the correctness of ordering, performance in doing so, and the reliability of all the modules, in this priority based transaction ordering block chain network. The validation of the core components like priority ordering and queuing of the transactions based on the priority, ensuring accurate transaction insertion, extraction, and ordering of transactions. The testing framework also includes the verification of gas fee computation across the varying loads on the network, and the congestion of the network based on the number of transactions in the mempool. The transaction validation logic and the isolated behaviour of the smart contract functions are also tested for correctness with this methodology.

To assess the end-to-end transaction flow, integration testing was carried out from the client in through the priority gateway and the Hyperledger fabric network to the ledger,

mainly to assess and verify the correctness of the priority based ordering, concurrent transaction ordering, and its resilience to the faults under the fault conditions which include peer crashes or network partitions.

Performance testing was also carried out to measure the system behaviour under varying operational loads. Load tests sustained 100 transactions per second(TPS), stress tests conducted to test and evaluate the systems response to loads beyond its capacity. Latency tests tracked the time taken for transaction confirmation across various priority levels and across various blocks, and mempool extremities tests were conducted to observe its behaviour under high transaction volumes.

Several test scenarios were designed to emulate real-world transaction distributions, including the ideal conditions of equal, then skewed, burst, and finally a gradual increase in traffic as patterns. The key performance metrics that were targeted included throughput, consistency in the ordering of transactions according to priority, latency depending on the number of transactions or traffic, gas fee calculation results, and overall stability of system during high load conditions. These test conditions collectively ensure that the system meets the required standards of efficiency and fairness, while being subjected to varying workloads.

6.10 Deployment Strategy

The current deployment framework is built on a local docker compose setup and involves the use of Hyperledger Fabrics test network, providing an ideal environment for testing as it involves the use of a single powerful single processing environment that is ideal for rapid development and testing.

This setup allows configuration and efficient redeployment of chaincode with minimal overhead or issues, which enables the quick validation of any new features that have been added or transaction logic that has been changed. In terms of scalability, the architecture has been designed with a clear cut path towards expansion or a staging environment, which involves multi virtual machine deployment that is hosted on larger scale platforms like AWS or GCP, including priority gateway instances, load balancing during increased traffic, and monitoring of the system for reliability under simulated workloads.
An ambitious future deployment involving a production grade deployment would include leveraging kubernetes and hyperledger operators for the automated managing, scaling, and fault tolerance of the system. Further improvements would involve DB replication for increased availability, and CDN distribution for client APIs. The deployment pipeline

would include a structured CI/CD process from normal git commits to full production rollouts. While the current deployment is limited and remains small scale for controlled testing and development of the network, it provides a base framework that can be scaled up to an enterprise grade blockchain network.

The transactions are first added to the mempool which are prioritized based on each of their type, giving swap transactions the highest priority and transfer the lowest. The transactions are then batched into five different blocks, each one of them containing five committed transactions, forming the validated part of the ledger. The validation process ensures that all the included transactions are successfully executed and marked as committed. Of the 30 transactions, 25 are validated and committed, where the other 5 remain pending in the mempool waiting to be included in the future blocks. The pending transactions represent the current network activity and simulate realistic blockchain throughput and queuing behavior under limited batch capacity.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Ledger Summary (Simulation Mode)


```
{
  "ledger_summary": {
    "total_transactions": 30,
    "validated_blocks": 5,
    "committed_transactions": 25,
    "pending_transactions": 5,
    "mempool_distribution": { "0": 1, "1": 1, "2": 1, "3": 2
},  "batching_strategy": "priority-based",
    "execution_mode": "simulation"
  }
}
```


8.2 Complete Network State


```
{
  "mempool": {
    "pending": [
      { "id": "TX26", "from": "A26", "to": "B26", "amount": "40", "type": "borrow", "priority": 1, "status":
"pending" },
      { "id": "TX27", "from": "A27", "to": "B27", "amount": "22", "type": "lend", "priority": 2, "status": "pending"
},
      { "id": "TX28", "from": "A28", "to": "B28", "amount": "15", "type": "transfer", "priority": 3, "status":
"pending" },
      { "id": "TX29", "from": "A29", "to": "B29", "amount": "10", "type": "transfer", "priority": 3, "status":
"pending" },
      { "id": "TX30", "from": "A30", "to": "B30", "amount": "18", "type": "swap", "priority": 0, "status": "pending"
}
    ],
    "maxSize": 100,
    "byPriority": { "0": 1, "1": 1, "2": 1, "3": 2 }
  },
  "blocks": [
    {
      "block_id": 1,
      "transactions": [
        { "id": "TX01", "type": "swap", "priority": 0 },
        { "id": "TX02", "type": "swap", "priority": 0 },
        { "id": "TX03", "type": "swap", "priority": 0 },
        { "id": "TX04", "type": "swap", "priority": 0 },
        { "id": "TX05", "type": "swap", "priority": 0 }
      ],
      "validation_status": "committed",
      "block_hash": "a37fbd44ae"
    },
```

```
    {
      "block_id": 2,
      "transactions": [
        { "id": "TX06", "type": "swap", "priority": 0 },
        { "id": "TX07", "type": "swap", "priority": 0 },
        { "id": "TX08", "type": "borrow", "priority": 1 },
        { "id": "TX09", "type": "borrow", "priority": 1 },
        { "id": "TX10", "type": "borrow", "priority": 1 }
      ],
      "validation_status": "committed",
      "block_hash": "c18db2145b"
    },
    {
      "block_id": 3,
      "transactions": [
        { "id": "TX11", "type": "borrow", "priority": 1 },
        { "id": "TX12", "type": "borrow", "priority": 1 },
        { "id": "TX13", "type": "lend", "priority": 2 },
        { "id": "TX14", "type": "lend", "priority": 2 },
        { "id": "TX15", "type": "lend", "priority": 2 }
      ],
      "validation_status": "committed",
      "block_hash": "b90adf9c33"
    },
    {
      "block_id": 4,
      "transactions": [
        { "id": "TX16", "type": "lend", "priority": 2 },
        { "id": "TX17", "type": "lend", "priority": 2 },
        { "id": "TX18", "type": "transfer", "priority": 3 },
        { "id": "TX19", "type": "transfer", "priority": 3 },
        { "id": "TX20", "type": "transfer", "priority": 3 }
      ],
      "validation_status": "committed",
      "block_hash": "f29e6b91cd"
    },
    {
      "block_id": 5,
      "transactions": [
        { "id": "TX21", "type": "transfer", "priority": 3 },
        { "id": "TX22", "type": "transfer", "priority": 3 },
        { "id": "TX23", "type": "transfer", "priority": 3 },
        { "id": "TX24", "type": "transfer", "priority": 3 },
        { "id": "TX25", "type": "transfer", "priority": 3 }
      ],
      "validation_status": "committed",
      "block_hash": "d44c1a0b9f"
    }
  ],
  "ledger_summary": {
    "total_transactions": 30,
```

8.3 Block Visualisation

```
PriorityFabric Ledger
├── Block #1 [priority-based]
│ ├── TX01 (swap, p0)
│ ├── TX02 (swap, p0)
│ ├── TX03 (swap, p0)
│ ├── TX04 (swap, p0)
│ └── TX05 (swap, p0)
│
├── Block #2 [priority-based]
│ ├── TX06 (swap, p0)
│ ├── TX07 (swap, p0)
│ ├── TX08 (borrow, p1)
│ ├── TX09 (borrow, p1)
│ └── TX10 (borrow, p1)
│
├── Block #3 [priority-based]
│ ├── TX11 (borrow, p1)
│ ├── TX12 (borrow, p1)
│ ├── TX13 (lend, p2)
│ ├── TX14 (lend, p2)
│ └── TX15 (lend, p2)
│
├── Block #4 [priority-based]
│ ├── TX16 (lend, p2)
│ ├── TX17 (lend, p2)
│ ├── TX18 (transfer, p3)
│ ├── TX19 (transfer, p3)
│ └── TX20 (transfer, p3)
│
├── Block #5 [priority-based]
│ ├── TX21 (transfer, p3)
│ ├── TX22 (transfer, p3)
│ ├── TX23 (transfer, p3)
│ ├── TX24 (transfer, p3)
│ └── TX25 (transfer, p3)
│
└── Pending in Mempool (5)
├── TX26 (borrow, p1)
├── TX27 (lend, p2)
├── TX28 (transfer, p3)
├── TX29 (transfer, p3)
└── TX30 (swap, p0)
```

8.4 Mempool-Pending Transactions Table

TxID	From	To	Type	Priority	Status	Timestamp (UTC)
TX-26	userE	userA	borrow	1	pending	2025-11-09T12:10:45Z
TX-27	userB	userC	lend	2	pending	2025-11-09T12:11:02Z
TX-28	userF	userE	transfer	3	pending	2025-11-09T12:11:18Z
TX-29	userC	userB	swap	0	pending	2025-11-09T12:11:31Z
TX-30	userA	userD	lend	2	pending	2025-11-09T12:11:47Z
TX-31	userG	userH	transfer	3	pending	2025-11-09T12:12:05Z
TX-32	userB	userF	borrow	1	pending	2025-11-09T12:12:22Z
TX-33	userE	userG	swap	0	pending	2025-11-09T12:12:39Z
TX-34	userC	userH	lend	2	pending	2025-11-09T12:12:55Z
TX-35	userA	userF	transfer	3	pending	2025-11-09T12:13:12Z

8.5 Analysis

The transactions are first added to the mempool which are prioritized based on each of their types, giving swap transactions the highest priority and transfer the lowest. The transactions are then batched into five different blocks, each one of them containing five committed transactions, forming the validated part of the ledger. The validation process ensures that all the included transactions are successfully executed and marked as committed. Of the 30 transactions, 25 are validated and committed, where the other 5 remain pending in the mempool waiting to be included in the future blocks. The pending transactions represent the current network activity and simulate realistic blockchain throughput and queuing behavior under limited batch capacity.

CHAPTER 9
CONCLUSION AND FUTURE WORK

The project “Optimised Blockchain Network for Use-Case Specific Transaction” demonstrates a more smarter, fair and purpose driven alternative for the existing conventional blockchain networks. It ensures critical transactions such as swaps, borrowing and lending are processed with higher efficiency. The model is designed to tackle limitations like gas-fee-dependent prioritization, extensive transaction delays followed with starvation, by integrating priority based mempool, quota driven batching and congestion-aware gas fee implementation.

Using Hyperledger Fabric with RAFT consensus, the model ensures conclusive ordering, fault tolerance, security and transparency without sacrificing real world enterprise blockchain requirements. The smart contract developed using Fabric Contract API handles wallet structure, balance updates, gas fee computation and atomic state transitions throughout every transaction type. The single-channel architecture allows global visibility across peers, making prioritization effective in the entire network.

A technically smarter dynamic gas fee model, introduces a fairer yet controlled fee mechanism that adapts to the network congestion. The system has fairness, improved transaction sequencing efficiency and DeFi-centric performance without compromising the security or the blockchain principles of immutability and decentralization.